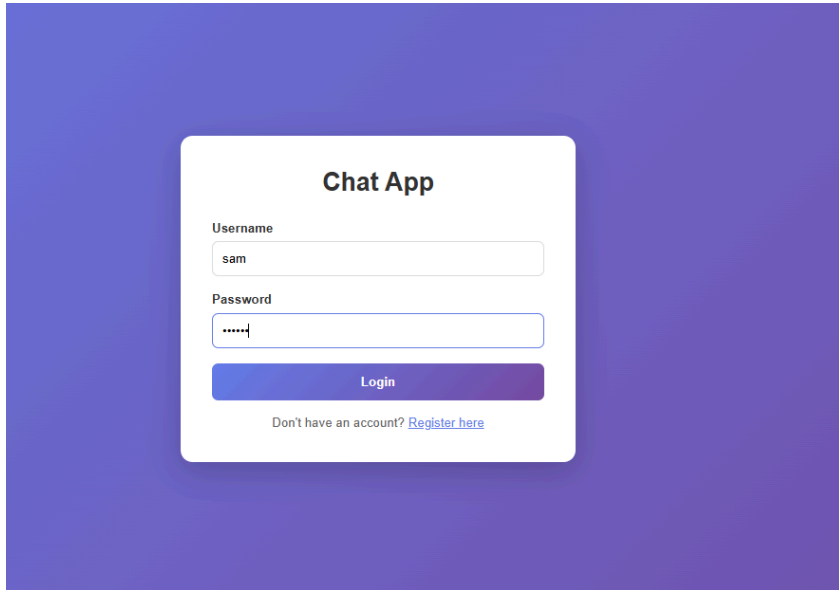


Login Page:

url:<http://127.0.0.1:8000/login.html>



The login page features a white card centered on a purple gradient background. The card is titled "Chat App" and contains a "Username" field with the text "sam", a "Password" field with masked characters ".....", and a "Login" button. Below the button is a link to "Register here" for users who don't have an account.

Chat App

Username
sam

Password
.....

Login

Don't have an account? [Register here](#)

Chat Page:

Url:<http://127.0.0.1:8000/chat.html>



The chat page has a dark green sidebar on the left with the title "Users" and a status "Disconnected". It includes a "Search users..." input and a "Search" button. The main area is titled "Select a user to chat" and contains a large beige box with the text "Select a user to start chatting". At the bottom, there is a "Type message..." input and a "Send" button.

Users
Disconnected

Search users... Search

Select a user to chat

Select a user to start chatting

Type message... Send

Search user:

Also tell user is active or not

User1:Sam

Users	Chat with tom
Disconnected tom Search tomtom@gmail.com	
	Type message...

User2:tom

Users

Connected

tom

Search

Chat with tom

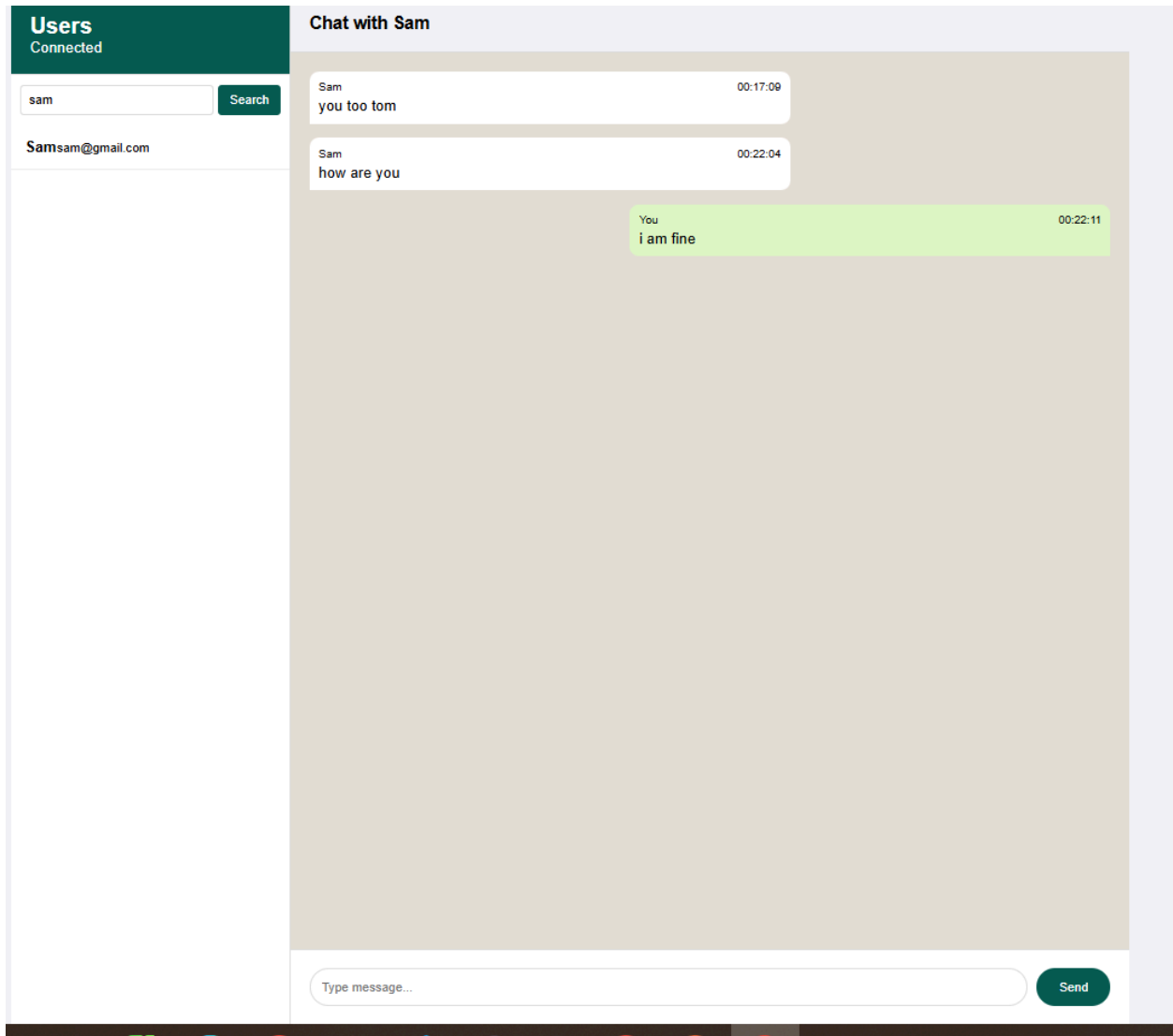
tom 00:18:51
good morning

You 00:22:04
how are you

tom 00:22:11
i am fine

Type message...

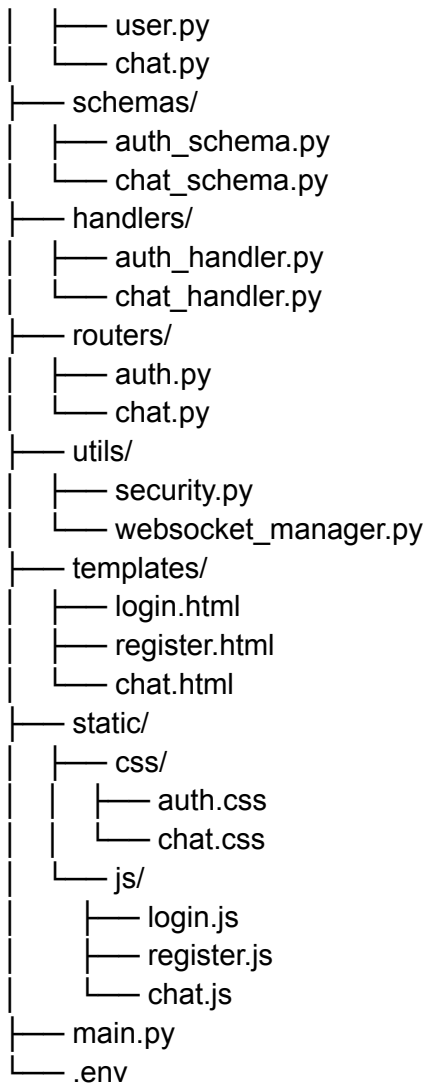
Send



Chat App Implementation

Create Project Structure

```
project/  
├── config/  
│   └── settings.py  
├── database/  
│   └── connection.py  
└── models/
```



1.2 Install Dependencies

```
uv init
```

```
uv add fastapi uvicorn websockets python-jose pddlib pydantic-settings
```

1.3 Create Environment File

File: `.env`

```
SECRET_KEY=your-secret-key-here
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60
DATABASE_URL=sqlite:///./chat.db
```

Database (database/connection.py)

```
engine = create_engine(DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, bind=engine)
Base = declarative_base()
```

```
def get_db():
    db = SessionLocal()
    try: yield db
    finally: db.close()
```

What: Creates DB connection, provides session

Models

models/user.py

```
class User(Base):
    __tablename__ = "users"
    id = Column(Integer, primary_key=True)
    username = Column(String, unique=True, nullable=False)
    email = Column(String, unique=True, nullable=False)
    hashed_password = Column(String, nullable=False)
```

models/chat.py

```
class ChatMessage(Base):
    __tablename__ = "chat_messages"
    id = Column(Integer, primary_key=True)
    sender_id = Column(Integer, ForeignKey("users.id"))
    receiver_id = Column(Integer, ForeignKey("users.id"))
    content = Column(Text, nullable=False)
    created_at = Column(DateTime, default=datetime.now)
```

What: Defines database tables

Utilities

utils/security.py

Important Lines:

```
pwd_hash = PasswordHash.recommended()
```

```
def verify_password(plain, hashed):
    return pwd_hash.verify(plain, hashed)

def get_password_hash(password):
    return pwd_hash.hash(password)

def create_access_token(data: dict):
    expire = datetime.now() + timedelta(minutes=60)
    data.update({"exp": expire})
    return jwt.encode(data, SECRET_KEY, algorithm=ALGORITHM)

def verify_token(token: str):
    return jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
```

What: Password hashing, JWT token creation/verification

utils/websocket_manager.py

Important Lines:

```
class ConnectionManager:
    def __init__(self):
        self.active_connections: Dict[int, WebSocket] = {}

    async def connect(self, websocket, client_id):
        await websocket.accept()
        self.active_connections[client_id] = websocket

    def disconnect(self, client_id):
        del self.active_connections[client_id]

    async def send_to_client(self, message, client_id):
        await self.active_connections[client_id].send_text(json.dumps(message))

manager = ConnectionManager()
```

What: Manages WebSocket connections

Handlers

handlers/auth_handler.py

Important Lines:

```
def register_user(db, username, email, password):
    hashed = get_password_hash(password)
```

```

new_user = User(username=username, email=email, hashed_password=hashed)
db.add(new_user)
db.commit()
return new_user

```

```

def login_user(db, username, password):
    user = db.query(User).filter(User.username == username).first()
    if not user or not verify_password(password, user.hashed_password):
        raise HTTPException(401, "Invalid credentials")
    token = create_access_token({"id": user.id, "username": user.username})
    return {"access_token": token, "user_id": user.id, "username": user.username}

```

What: User registration and login logic

handlers/chat_handler.py

Important Lines:

```

async def handle_chat_message(user_id, username, message, target_id, db):
    chat_message = ChatMessage(sender_id=user_id, receiver_id=target_id, content=message)
    db.add(chat_message)
    db.commit()
    await manager.send_to_client({"type": "message", "content": message}, target_id)

```

```

def get_chat_history(db, user1_id, user2_id):
    return db.query(ChatMessage).filter(
        (sender_id=user1_id, receiver_id=user2_id) | (sender_id=user2_id, receiver_id=user1_id)
    ).all()

```

```

def search_users(db, current_id, query):
    return db.query(User).filter(User.id != current_id, User.username.ilike(f"%{query}%")).all()

```

What: Save messages, get history, search users